

Assuming you are a data analyst/ scientist at Target, you have been assigned the task of analyzing the given dataset to extract valuable insights and provide actionable recommendations.

What does 'good' look like?

1. Data type of all columns in the "customers" table

 Filter Enter property name or value		
☐	Field name	Type
☐	customer_id	STRING
☐	customer_unique_id	STRING
☐	customer_zip_code_prefix	INTEGER
☐	customer_city	STRING
☐	customer_state	STRING

We can get the information from previewing the table. However, we can also use the following query to get the same information.

```
SELECT COLUMN_NAME, DATA_TYPE
```

```
FROM round-kit-462216-u9.Target_sql.INFORMATION_SCHEMA.COLUMNS
```

```
WHERE TABLE_NAME = 'Customers'
```

Result:

Job information		Results	Chart	JSON	Execution details
Row	COLUMN_NAME ▼	DATA_TYPE ▼			
1	customer_id	STRING			
2	customer_unique_id	STRING			
3	customer_zip_code_prefix	INT64			
4	customer_city	STRING			
5	customer_state	STRING			

Which can also be explained as

- customer_id: VARCHAR/CHAR (unique identifier)
- customer_unique_id: VARCHAR/CHAR (unique consumer identifier)
- customer_zip_code_prefix: VARCHAR/INTEGER (zip code prefix)
- customer_city: VARCHAR (city name)
- customer_state: VARCHAR (state code, e.g., 'SP')

2. Get the time range between which the orders were placed

Query:

```
SELECT MIN(order_purchase_timestamp) as earliest_order,
MAX(order_purchase_timestamp) as last_order
FROM `round-kit-462216-u9.Target_sql.Orders`
```

Result:

Job information		Results	Chart	JSON	Execution details	Execution graph
Row	earliest_order ▼	last_order ▼				
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC				

This gives the first and the last order placed between 2016 and 2018.

3. Count the Cities & States of customers who ordered during the given period

Query:

```
select count(distinct customer_city) as city,  
  
count(distinct customer_state) as state  
  
from `Target_sql.Customers` c  
  
join `Target_sql.Orders` o on c.customer_id = o.customer_id
```

Results:

Row	city	state
1	4119	27

Insite:

There are 4119 Cities and 27 States where customers placed the orders.

4. Is there a growing trend in the number of orders placed over the past years?

Query:

```
SELECT extract( year from order_purchase_timestamp) as order_year,  
  
count(*) as number_orders  
  
from `Target_sql.Orders`  
  
group by order_year  
  
order by order_year
```

Results:

Job information

Results

Chart

JSON

Execution details

Execution graph

Row	order_year	number_orders
1	2016	329
2	2017	45101
3	2018	54011

Insite: We see the orders jumped from 2016 to 2017 after 1 year of Target being in Brazil. From 2017 to 2018, we saw a 9% increase in the number of orders placed.

5. Can we see monthly seasonality in terms of the number of orders being placed?

Query:

```
SELECT extract(year from order_purchase_timestamp) as Order_Year,
extract(month from order_purchase_timestamp) as Order_Month,
count(*) as Total_count
from `Target_sql.Orders`
group by Order_Year, Order_Month
order by Order_Year, Order_Month
```

Results:

Row	Order_Year	Order_Month	Total_count
1	2016	9	4
2	2016	10	324
3	2016	12	1
4	2017	1	800
5	2017	2	1780
6	2017	3	2682
7	2017	4	2404
8	2017	5	3700

Job information Results Chart JSON Execution details Execution graph					
Row	Order_Year	Order_Month	Total_count		
9	2017	6	3245		
10	2017	7	4026		
11	2017	8	4331		
12	2017	9	4285		
13	2017	10	4631		
14	2017	11	7544		
15	2017	12	5673		
16	2018	1	7269		

17	2018	2	6728		
18	2018	3	7211		
19	2018	4	6939		
20	2018	5	6873		
21	2018	6	6167		
22	2018	7	6292		
23	2018	8	6512		

24	2018	9		16
25	2018	10	.	4

Insite:

The order count peaked in November 2017 (7,544 orders), with stable high volumes in 2018 (6,167–7,269 monthly). The November 2017 spike suggests strong holiday seasonality, while consistent 2018 performance indicates business scaling.

6. During what time of the day do Brazilian customers mostly place their orders?

Query:

```
SELECT case when extract(hour from order_purchase_timestamp) between 0 and 6 then
'Early_Morning'

when extract(hour from order_purchase_timestamp) between 7 and 12 then 'Morning'

when extract(hour from order_purchase_timestamp) between 13 and 18 then 'Afternoon'

else 'Night'

end as Time_of_day,

count(*) as order_count

from `Target_sql.Orders`

group by Time_of_day

order by order_count DESC
```

RESULTS:

Job information		Results	Chart	JSON	Execution details	Execution graph
Row	Time_of_day	order_count				
1	Afternoon	38135				
2	Night	28331				
3	Morning	27733				
4	Early_Morning	5242				

INSITE:

We do see a gradual increase in orders throughout the day ,but the max orders are placed in the time frame of 13:00 hours to 18:00 hours (from 1pm to 6pm).

7. Get the month-on-month number of orders placed in each state

Query:

```
SELECT c.customer_state,
extract (Year from order_purchase_timestamp) as Order_year,
extract(Month from order_purchase_timestamp) as Order_month,
count(*) as order_count
from `Target_sql.Orders` o
Join `Target_sql.Customers` c on o.customer_id = c.customer_id
group by c.customer_state,Order_year, Order_month
order by c.customer_state, Order_year, Order_month
limit 10;
```

RESULTS:

Row	customer_state	Order_year	Order_month	order_count
1	AC	2017	1	2
2	AC	2017	2	3
3	AC	2017	3	2
4	AC	2017	4	5
5	AC	2017	5	8
6	AC	2017	6	4
7	AC	2017	7	5
8	AC	2017	8	4
9	AC	2017	9	5
10	AC	2017	10	6

INSITE:

The following query shows the orders placed in each month according to the states and year.

8. How are customers distributed across all the states? (Unique Customers in each state)

Query:

```
SELECT c.customer_state,
count(distinct c.customer_unique_id) as unique_customers
from `Target_sql.Customers` c
join `Target_sql.Orders` o on c.customer_id = o.customer_id
group by c.customer_state
order by unique_customers DESC
limit 10;
```

RESULTS:

Row	customer_state	unique_customers
1	SP	40302
2	RJ	12384
3	MG	11259
4	RS	5277
5	PR	4882
6	SC	3534
7	BA	3277
8	DF	2075
9	ES	1964
10	GO	1952

INSITE: We found the top 10 states with the most unique customers, with SP having the most customers at 40,302.

9. Get the % increase in the cost of orders from 2017 to 2018 (Jan to Aug only)

Query:

10. Calculate the Total & Average value of order price for each state

Query:

```
SELECT c.customer_state,
round(sum(oi.price), 2) as total_price,
round(avg(oi.price), 2) as average_price
from `Target_sql.Order_items` oi
join `Target_sql.Orders` o on oi.order_id = o.order_id
join `Target_sql.Customers` c on o.customer_id = c.customer_id
group by c.customer_state
order by total_price DESC
limit 10;
```

RESULTS:

Row	customer_state	total_price	average_price	
1	SP	5202955.05	109.65	
2	RJ	1824092.67	125.12	
3	MG	1585308.03	120.75	
4	RS	750304.02	120.34	
5	PR	683083.76	119.0	
6	SC	520553.34	124.65	
7	BA	511349.99	134.6	
8	DF	302603.94	125.77	
9	GO	294591.95	126.27	
10	ES	275037.31	121.91	

INSITE: The following query gives us the top 10 states with the most order value with the average order value of the state. SP is the state with highest order value

11. Calculate the Total & Average value of order freight for each state

Query:

```
SELECT c.customer_state,
round(sum(oi.freight_value),2) as total_freight,
round(avg(oi.freight_value), 2) as average_freight
from `Target_sql.Order_items` oi
join `Target_sql.Orders` o on oi.order_id = o.order_id
join `Target_sql.Customers` c on o.customer_id = c.customer_id
group by c.customer_state
order by total_freight DESC
limit 10;
```

RESULTS:

Row	customer_state	total_freight	average_freight
1	SP	718723.07	15.15
2	RJ	305589.31	20.96
3	MG	270853.46	20.63
4	RS	135522.74	21.74
5	PR	117851.68	20.53
6	BA	100156.68	26.36
7	SC	89660.26	21.47
8	PE	59449.66	32.92
9	GO	53114.98	22.77
10	DF	50625.5	21.04

INSITE: The query gives us the total freight value as per the state with SP again being the highest state with the total freight.

12. Find the number of days taken to deliver each order and the difference between estimated & actual delivery date

Query:

```
SELECT order_id,

date_diff(order_delivered_customer_date, order_estimated_delivery_date, day) as
diff_estimated_date,

date_diff(order_delivered_customer_date ,order_purchase_timestamp, day) as
delivery_days

from `Target_sql.Orders`

where order_delivered_customer_date is not null

and order_estimated_delivery_date is not null

and order_purchase_timestamp is not null

order by order_id DESC

limit 10;
```

RESULTS:

Row	order_id	diff_estimated_d...	delivery_days
1	fffe41c64501cc87c801fd61db3...	-13	5
2	fffe18544ffabc95dfada21779c...	-8	1
3	fffce4705a9662cd70adb13d4a...	-12	4
4	fffc46ef2263f404302a634eb5...	-8	9
5	fffc94f6ce00a00581880bf54a7...	-7	17
6	fffbec3b5462987e66fb49b1c54...	-17	16
7	fffb9224b6fc7c43ebb0904318b...	-9	21
8	fffb2ef8874127f75b52b643880...	-10	17
9	fffb0b1a50e65c449020434fa83...	2	34
10	fffa82886406ccf10c7b4e35c4f...	-15	21

Insite: We can see in the 10 rows that we rarely see any delays. The delivery happens before the estimated time.

Analysis Based on Payments (10 points)

13. Month-on-month number of orders placed using different payment types

Query:

```
SELECT p.payment_type,
extract(Year from o.order_purchase_timestamp) as order_year,
extract(month from o.order_purchase_timestamp) as order_month,
count(distinct p.order_id) as order_number
from `Target_sql.Payments` p
join `Target_sql.Orders` o on p.order_id = o.order_id
group by p.payment_type, order_year, order_month
order by p.payment_type, order_year, order_month
```

RESULTS:

Row	payment_type	order_year	order_month	order_number
1	UPI	2016	10	63
2	UPI	2017	1	197
3	UPI	2017	2	398
4	UPI	2017	3	590
5	UPI	2017	4	496
6	UPI	2017	5	772
7	UPI	2017	6	707
8	UPI	2017	7	845
9	UPI	2017	8	938
10	UPI	2017	9	903
11	UPI	2017	10	993

Insite: The following query will give the details of orders placed in every month with different payment types. The above is a sample put put for the same.

14. Number of orders placed based on payment installments

Query:

```
SELECT p.payment_installments,
count(distinct p.order_id) as order_count
from `Target_sql.Payments` p
join `Target_sql.Orders` o on p.order_id = o.order_id
group by p.payment_installments
order by p.payment_installments
```

RESULTS:

Row	payment_installm...	order_count
1	0	2
2	1	49060
3	2	12389
4	3	10443
5	4	7088
6	5	5234
7	6	3916

8	7		1623
9	8		4253
10	9		644

Insite: The following query gives us the number of orders that were purchased as installments with grouping of the orders based on the installments selected.

15. Top 5 states where order delivery is really fast compared to the estimated date

Query:

```
SELECT c.customer_state,

avg(date_diff(o.order_delivered_customer_date, o.order_estimated_delivery_date, day))
as Average_delivery_diff

from `Target_sql.Orders` o

join `Target_sql.Customers` c on o.customer_id = c.customer_id

where o.order_delivered_customer_date is not null

and o.order_estimated_delivery_date is not null

group by c.customer_state

order by Average_delivery_diff DESC

limit 5;
```

Results:

Row	customer_state	Average_delivery_diff
1	AL	-7.94710327455919
2	MA	-8.768479776847979
3	SE	-9.173134328358211
4	ES	-9.6185463659147743
5	BA	-9.9348894348894348

Insite: We can see that AL is the state where the delivery happens fastest in the top 5 states.

16. Top 5 states with the highest & lowest average freight value

Query:

```
with Freight as (select c.customer_state,
round(avg(ot.freight_value), 2) as Average_freight
from `Target_sql.Order_items` ot
join `Target_sql.Orders` o on ot.order_id = o.order_id
join `Target_sql.Customers` c on o.customer_id = c.customer_id
group by c.customer_state
)
(select customer_state, Freight.Average_freight
from Freight
order by Freight.Average_freight DESC
limit 5)
UNION ALL
(select customer_state, Freight.Average_freight
from Freight
order by Freight.Average_freight
limit 5);
```

Results:

TOP 5 Highest

1	RR	42.98
2	PB	42.72
3	RO	41.07
4	AC	40.07
5	PI	39.15

TOP 5 Lowest

6	SP	15.15
7	PR	20.53
8	MG	20.63
9	RJ	20.96
10	DF	21.04

Insite: We can see the difference between the top 5 and the bottom 5 states. The high freight might be because of the distance between the hub and the locations of the orders.

17. Top 5 states with the highest & lowest average delivery time

Query:

```
with Delivery_time as (select c.customer_state,

round(avg(date_diff(o.order_delivered_customer_date, o.order_purchase_timestamp,
day))) as Average_delivery

from `Target_sql.Orders` o

join `Target_sql.Customers` c on o.customer_id = c.customer_id

where o.order_delivered_customer_date is not null

and o.order_purchase_timestamp is not null

group by c.customer_state

)

(select customer_state, Delivery_time.Average_delivery

from Delivery_time

order by Delivery_time.Average_delivery DESC

limit 5)

UNION ALL

(select customer_state, Delivery_time.Average_delivery

from Delivery_time
```

```
order by Delivery_time.Average_delivery
```

```
limit 5);
```

Results:

Top 5 Highest delivery times

Row	customer_state	Average_delivery
1	RR	29.0
2	AP	27.0
3	AM	26.0
4	AL	24.0
5	PA	23.0

Top 5 Lowest delivery times

6	SP	8.0
7	MG	12.0
8	PR	12.0
9	DF	13.0
10	SC	14.0

Insite: We see the time for delivery is higher in RR state and the lowest is in SP.